WHO WROTE IT?

Identifying LLMs from Their Responses

CMPSC 448 Midterm Project
Team leader: Kirubel Tamrat
Team member: Chukwubikem Dara
October 11, 2026

Study: Three model families, 100 shared prompts, two neural-network classifiers, and four research questions.

1. Problem and motivation

When we read an AI-generated answer, we usually cannot tell which model wrote it. We wanted to test whether the wording and structure of an answer give us any useful clues. We treated this as a three-class classification problem: from a response, predict ChatGPT, Claude, or Gemini. We also wanted to rule out a shortcut: learning the kinds of questions instead of learning anything about the responses.

Our main questions were whether a CNN and an LSTM could recognize the source model from a response (**RQ1**) and whether providing the user prompt would help (**RQ2**). We also tested whether the classifiers worked on a new task category (**RQ3**) and what simple writing characteristics might distinguish the models (**RQ4**).

2. Dataset and preparation

We used the public *LLM Response Evaluation Benchmark* from Pranavsakunde on GitHub (cited at the end). It contains 100 shared prompts across general knowledge (20), coding (15), mathematics (15), writing (15), reasoning (15), translation (10), and safety (10). The publisher reports collecting an answer from ChatGPT, Claude, and Gemini for every prompt. We used the prompts and response text, not the source dataset's evaluation scores. Our team did not generate the original responses.

We checked for blank responses and retained **300 answers** from **100 shared prompts**. Each model has 100 answers (ChatGPT: 100, Claude: 100, Gemini: 100). We split whole prompts into 69 training groups, 16 validation groups, and 15 test groups. There are no shared prompts between splits. Our test set has 45 answers, 15 per model. The benchmark author describes the responses as authentic, but we did not have model API logs to independently verify their origins.

We preserved the response text and visible formatting apart from trimming leading or trailing whitespace. For **prompt-only**, the input is the question; for **response-only**, it is the answer; for **combined**, it is question, separator, then answer. We lowercased and tokenized words, numbers, and punctuation. The vocabulary was built from training examples only, which keeps unseen test vocabulary from leaking into the model.

3. Models and training procedure

CNN: A trainable word embedding feeds three one-dimensional convolution branches, with kernel widths 3, 5 and 7. Max pooling turns each branch into a fixed-length vector, and dropout plus a fully connected layer produce three scores. The different kernel widths are intended to capture short patterns such as repeated phrases, punctuation and formatting conventions.

LSTM: The same embedding idea feeds a bidirectional long short-term memory network. Forward and backward final hidden states are concatenated, then passed through dropout and a classification layer. Unlike the CNN, the LSTM processes tokens in sequence and can potentially use longer-range ordering.

Both classifiers are trained from scratch with cross-entropy loss and Adam (learning rate 0.002, weight decay 0.0001). The default batch size is 32, the sequence limit is 256 tokens, embeddings are 64-dimensional, and the CNN channels / LSTM hidden size are 48. Training uses up to 14 epochs, gradient clipping, and early stopping on validation loss (patience 5). The random seed is 42. The best validation checkpoint is evaluated once on the held-out test prompts. CNN filters have widths 3, 5 and 7; both models use dropout 0.3. Prompt + response has the same 256-token cap; placing the prompt first can push the end of the answer outside that window. This is one reason the combined condition is not guaranteed to help.

Reported metrics are test accuracy and macro-F1. Accuracy is easy to interpret, but macro-F1 gives equal weight to each model family. With three balanced classes, random chance is approximately 33.3%. The small dataset means the results should be treated as an initial experiment, not a reliable real-world authorship detector.

4. Experiments and results

Model	Features	Test accuracy	Macro-F1	Epochs
CNN	input	33.3%	0.314	11
CNN	output	80.0%	0.797	14
CNN	combined	75.6%	0.757	14
LSTM	input	33.3%	0.291	12
LSTM	output	55.6%	0.557	14
LSTM	combined	51.1%	0.484	11

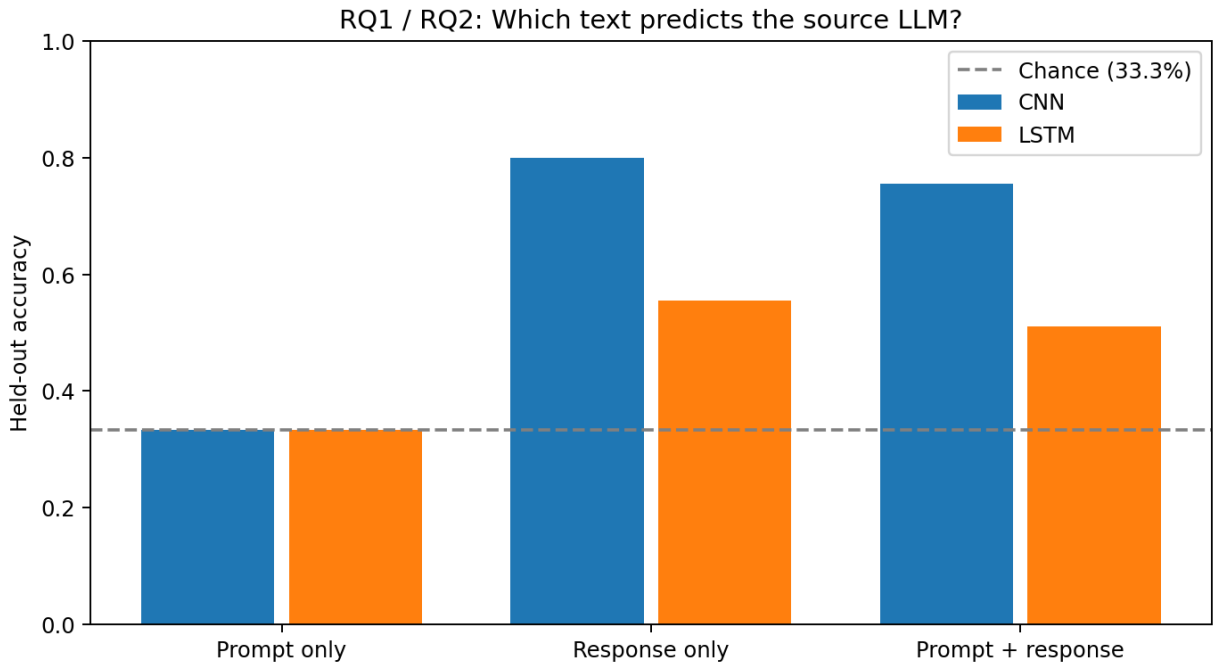


Figure 1. Same train/validation/test prompt groups are used across all six CNN and LSTM conditions.

RQ1: With the responses alone, the CNN scored 80.0% accuracy (macro-F1 0.797), compared with 55.6% (macro-F1 0.557) for the LSTM. The CNN correctly labeled 14 of 15 ChatGPT answers and 11 of 15 Claude answers; it also correctly labeled 11 of 15 Gemini answers. Its mistakes included confusing Claude with Gemini. The numbers suggest identifiable patterns in this dataset, not a universal way to prove authorship.

RQ2, CNN: Prompt-only scored 33.3%, response-only 80.0%, and prompt + response 75.6%. Adding the prompt was 4.4 percentage points worse than using the response alone. Every prompt appears with all three labels, so identical prompt text cannot uniquely identify a model. The chance-level prompt result is therefore a useful sanity check, not a meaningful predictor.

RQ2, LSTM: Prompt-only scored 33.3%, response-only 55.6%, and prompt + response 51.1%. Adding the prompt was 4.4 percentage points worse than using the response alone. Every prompt appears with all three labels, so identical prompt text cannot uniquely identify a model. The chance-level prompt result is therefore a useful sanity check, not a meaningful predictor.

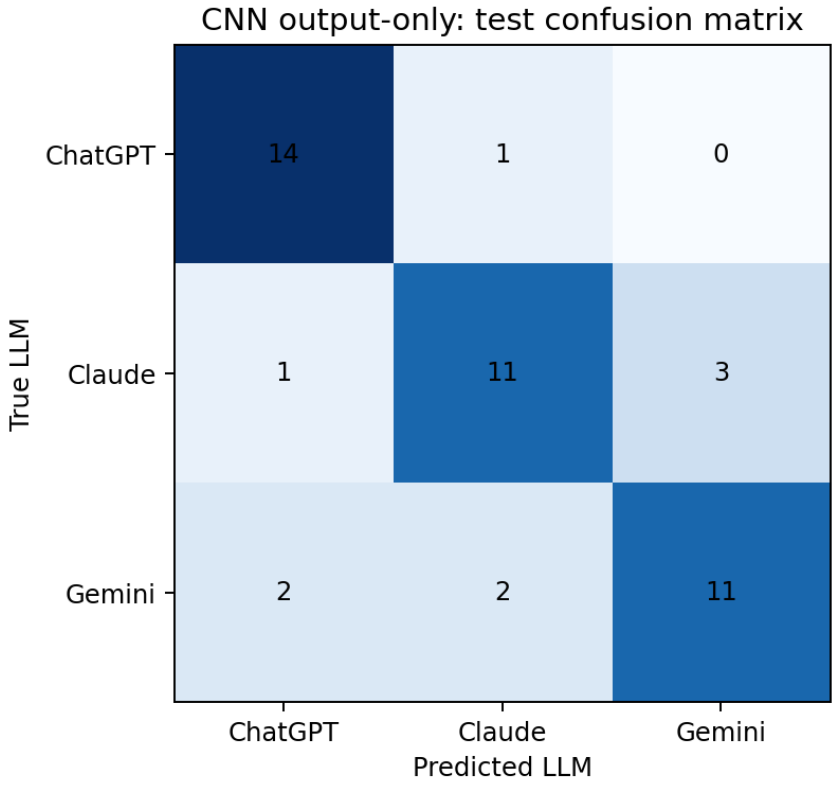


Figure 2. CNN response-only errors, broken down by true and predicted family.

5. Additional analyses

RQ3: generalization to a different kind of question. We held out the writing category completely from training and validation. Models were trained on all remaining categories, then tested on writing prompts they had never seen. This checks whether a classifier has learned patterns that travel beyond the types of questions in its training set.

Model	Held-out writing prompts	Test accuracy	Macro-F1
CNN	15	42.2%	0.338
LSTM	15	53.3%	0.524

Compared with the usual response-only test (CNN 80.0%, LSTM 55.6%), the leave-writing-out scores (42.2%, 53.3%) show a much bigger drop for the CNN. This suggests the CNN may depend more on patterns tied to familiar tasks. However, the test sets and training data are different across the two experiments, so this is an illustration of domain shift rather than a perfectly controlled comparison. There are only 15 writing prompts.

RQ4: What makes the output look different? We measured response length, average word length, lexical diversity (unique-word ratio), average sentence length, Markdown symbol count, bullet lines, and newlines. These are descriptive properties rather than proof of what the neural network learned.

Family	Words	Lexical diversity	Markdown marks	Bullets
ChatGPT	104.3	0.628	0.9	0.2
Claude	80.7	0.730	0.5	0.0
Gemini	38.5	0.813	0.1	0.0

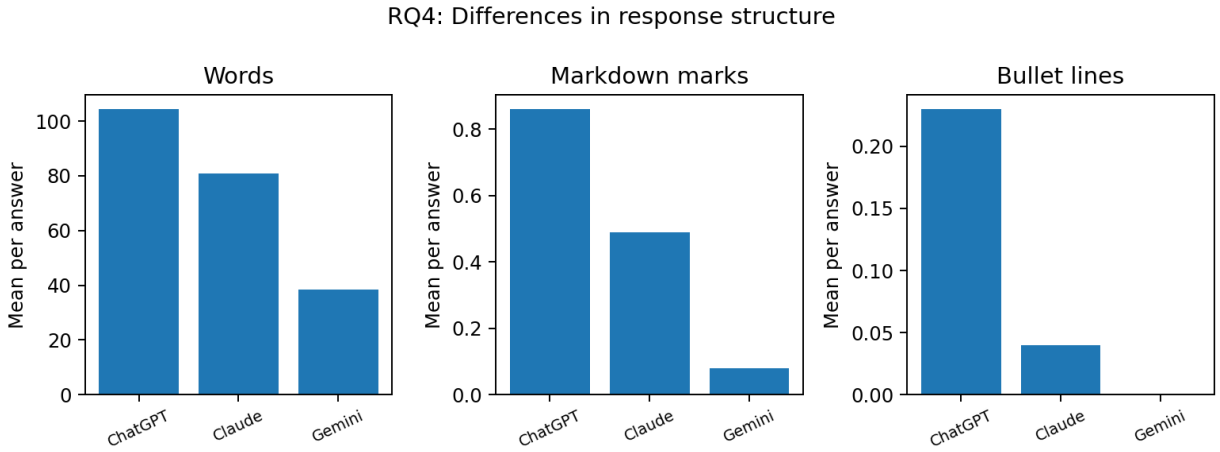


Figure 3. Average response structure. Features have different scales and should not be compared as if they share units.

To make the formatting comparison more concrete, we kept the trained response-only models fixed and stripped Markdown/list characters from the test answers. This is a *test-time perturbation*, not a new training run, so it isolates how sensitive the existing predictions are to these visible cues.

Model	Original test	Stripped test	Change
CNN	80.0%	77.8%	-2.2 pp
LSTM	55.6%	57.8%	+2.2 pp

We also trained a **simple logistic regression control** on response-style measurements. It uses the exact same prompt-disjoint train/test groups, but it does not read the text itself. This helps put the neural-network results into context: if a few numbers already identify a model, the neural networks may be using some of these simpler signals too.

Style-only control	Test accuracy	Macro-F1
Response word count	48.9%	0.446

Style-only control	Test accuracy	Macro-F1
Seven response-style features	64.4%	0.645

Response length alone did better than chance, and the seven-feature model did better still. We cannot conclude these are the same signals learned by the neural networks. Also, short answers tend to have a higher unique-word ratio, so differences in lexical diversity should not be interpreted independently of length. Stripping markup changed the CNN by only one correct answer out of 45; formatting is part of the story, but it is unlikely to explain everything.

6. What this experiment shows, and its limits

One thing we learned is that a sensible train/test split matters just as much as choosing a network. Because the same questions were asked of all three models, we split by the whole prompt rather than splitting response rows at random. Otherwise the exact same question could appear during training and testing, making the scores look stronger than they really are. The 33.3% prompt-only results helped check that the experiment was not simply learning the questions.

The clearest result was that the CNN beat the other network on response-only classification (80.0% versus 55.6%). That does not mean CNNs are always better for language tasks: we trained both networks from scratch on just 207 examples, and this is one 45-answer test split. If we repeated the project, we would rerun it with several seeds and report the mean and variability, rather than lean on one headline accuracy.

The combined-input result was also useful: supplying more text did not automatically improve predictions. Our 256-token cap means the prompt uses space that would otherwise hold response tokens. We would test a longer sequence limit or separate prompt/response encoders next. Finally, the style-only baseline made us question whether a model was recognizing genuine linguistic habits or more obvious signals, especially answer length. A length-matched dataset would help separate those explanations.

There are limits to what we can claim: the benchmark has just 100 prompts, the source model names come from its publisher rather than API logs we could independently inspect, and exact model versions and generation settings are not fully documented. Our classifiers only choose among three known labels, even for text from another source. These results should not be used to accuse someone of AI authorship. More prompts, new model versions, multiple random splits, and a separate independent dataset would give a stronger test.

7. Reproducibility and source

The accompanying repository files contain preprocessing, CNN/LSTM implementations, prompt-group splitting, training and evaluation, RQ3/RQ4 analysis, figure generation, and this PDF report builder. The seed, model configuration, trained output-only checkpoints, split audit and output JSON are recorded in the results directory. Running the scripts with a verified source workbook should regenerate the numerical tables.

Dataset citation: Pranavsakunde, *LLM Response Evaluation Benchmark*, GitHub: https://github.com/Pranavsakunde/LLM-Response-Evaluation-Benchmark. The public README says the 100 prompts were answered by ChatGPT, Claude and Gemini. The original response workbook is not included in our project repository; public access does not automatically grant redistribution rights.

Course specification: CMPSC 448, *Who Wrote It? Identifying LLMs from Their Responses*, modified September 17, 2026, provided assignment PDF.